

Chapter 4. Models Behind the Agents: Capabilities and Optimization

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mcronin@oreilly.com.

By now you have already done the hard work to understand how to design AI agents and make them better. This is the right moment for me to ask you to step back, look behind the scenes, and think more deeply about the models that make them intelligent. You might ask yourself why I didn’t introduce the models at the beginning. There is a reason. Let me tell you.

For this chapter it’s crucial to understand what you can do with your AI agents, because it completes the narrative arc of how I want you to think about them. I want you to think about your agents as if you were hiring team members. To do that well, you need the bigger picture: how your team interacts, how you structure it, and how you coach it to improve and collaborate. All of this was covered in the previous chapters.

This is my own mental model for designing AI agents. I think of agents as dynamic, collaborative systems rather than isolated components. Every model size and model family has a distinct skill profile. If you build your multi-agent system with that in mind, you’ll not only get better results, you’ll also save money. Let’s think of it a different way. You have learned how to manage your team; now it’s time to learn how to hire team members. In this chapter you learn how you decide who to onboard to the team, what skills they bring, and why that matters for performance, improvement, and cost.

Just as in human teams, each member has different strengths, weaknesses, and ways of working, and their skills complement one another. Same

holds true with AI Agents, when you combine them thoughtfully, they achieve much more together than any single model could on its own. Once you understand their strengths and how they can collaborate, you can assign the right tasks and unlock the full potential of your system.

However, it's important to know that this chapter isn't meant as an introduction to the transformer architectures or to parameter-efficient fine-tuning (PEFT) methods. I assume you've seen those before. What I want to do here is refresh that knowledge and line it up with the agentic mindset, so it becomes obvious why model choice, context strategy, and light-weight specialization (LoRA, adapters, quantized bases) matter in the context of orchestrating multiple agents.

To understand this better, you'll examine different architectures under this light, compare open weight and closed API models. In addition, you'll map context window tradeoffs to deployment strategies, and understand when specialization through fine-tuning or adapters pays off. By the end, you'll know how to staff your team, assign the right tasks, and optimize for both performance and spend. Let's meet the candidates, understand their strengths, and hire well.

The Big Picture: Choosing the Right Architecture

Modern transformer variants are designed to handle particular workloads and deployment needs. The right choice depends on your objective, how information flows through the system, and the constraints you face in serving or scaling your models. Each architecture (encoder-only, decoder-only, encoder-decoder, or even MoE) has a unique role to play. Selecting the right mix means aligning each model's strengths with your goals, computational budget, and serving constraints to achieve the best balance between capability and efficiency. [Table 4-1](#) gives you a high-level understanding of each architecture and capability.

Table 4-1. Transformer Architecture Types

Type	Primary capability	Typical use cases
Encoder-only	Representation learning. Produces contextual embeddings for input tokens.	Text classification. Semantic search. Named entity recognition (NER).
Decoder-only	Autoregressive generation. Learns to predict the next token given previous context.	Text generation. Code synthesis. Conversational agents and chat systems.
Encoder-decoder	Sequence-to-sequence mapping between input and output representations.	Translation. Summarization. Question answering over provided context.
Embedding models	Produce dense or sparse vector representations for efficient retrieval and similarity comparison.	Retrieval-augmented generation (RAG). Semantic similarity search. Clustering.
Mixture of Experts (MoE)	Sparse expert routing for scalable compute and specialization across tasks.	Efficient large-scale generation. Multitask learning. Adaptive model scaling.

Let’s take a deeper look into decoder-only, encoder-only and MoE models, since these are the backbones of modern AI systems.

Decoder-Only Models

Decoder-only models are what you usually think of when you hear “LLM”. They form the backbone of systems like GPT or Claude and follow a simple but powerful idea: generate one token at a time, using everything that came before as context.

You already know the structure: no encoder, no cross-attention, just a stack of transformer blocks predicting the next token again and again. This autoregressive (auto = self, regressive = based on prior values) setup means the model learns to look only backward in the sequence. Each new token depends on its own previous outputs, creating a feedback loop where every generation step builds on what came before. [Figure 4-1](#) gives you a high-level overview of the decoder-only architecture.

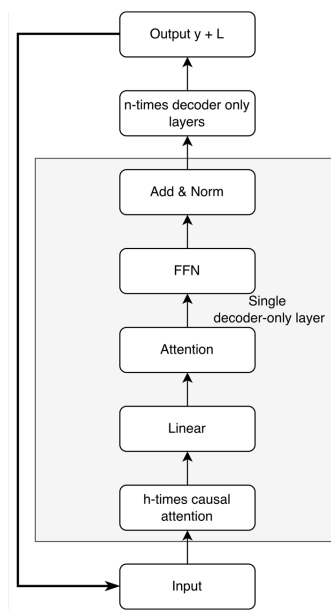


Figure 4-1. Overview of a decoder only architecture, where $\mathbf{y} + \mathbf{L}$ represents the final output transformation, meaning that the decoder's hidden state $\mathbf{y}$ is passed through the learned output layer $\mathbf{L}$ to produce the logits for the next token.

The fact that these models generate outputs autoregressively makes them brilliant for text and code generation, reasoning, and dialogue. In short, decoder-only models are your generators, your coders, your talkers, and your planners. They are the ones that think step by step, always building on their past thoughts. However, understanding how they handle that memory, literally and conceptually, is the key to making them faster and more efficient. Let's use math to see why. To enable this causal (autoregressive) attention, the self-attention mask enforces causality:

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{M}_{\text{causal}} \right) \mathbf{V}$$

This means every time the model generates a new token, it would have to recompute all previous attention states, because the model can attend only to tokens $\leq t$ at step t . However, since the previously computed $\mathbf{K}_{1:t-1}$ and $\mathbf{V}_{1:t-1}$ never change, they can safely be reused for step $t + 1$. That's where key-value (KV) caching comes in.

KV caching: why it matters

Key-value caching is a clever optimization that stores previously computed keys and values from attention layers so the model can reuse them instead of recalculating them from scratch. KV caching speeds up inference dramatically, but at a cost. As the sequence grows, the cache grows too, layer by layer, until memory consumption starts to dominate. The art lies in balancing speed and memory, which becomes critical when deploying models at scale or across multiple agents. As a rule of thumb, memory grows with sequence length $\times$ layers $\times$ heads. Overall, you get a throughput win at the cost of memory footprint. It's also important to understand that state-of-the-art (SOTA) models don't use the original attention mechanism anymore. They are using usually *Grouped-Query Attention (GQA)*, which instead of every query head having its own K and V projections,

groups multiple query heads to share keys and values, reducing KV cache size dramatically. In Hugging Face you can easily access your model’s architecture by first loading the model with `model = AutoModelForCausalLM.from_pretrained(model_name)` and then calling `model.config` this will print the whole architectural setup, for `Qwen2.5-7B-Instruct-1M` this gives this information about its GQA setup:

```
"num_attention_heads": 28,  
"num_key_value_heads": 4,
```

That Qwen 2.5 setup, 28 query heads sharing just 4 key-value heads, is the perfect modern example of GQA in practice. [Table 4-2](#) details how this affects memory.

Table 4-2. Grouped Query Attention Configuration (Example: Qwen 2.5)

Component	Value	Meaning
Q-heads	28	Independent query projections to preserve expressiveness.
K/V-heads	4	Shared key and value projections to minimize cache size and memory bandwidth.
Grouping ratio	$28 \div 4 = 7$	Each K/V pair serves 7 query heads. ~7× reduction in KV cache memory and bandwidth during decoding

Efficient KV cache management is essential for scaling your decoder-only models, as it directly impacts both memory footprint and responsiveness. Proper management sets the basis whether your system scales gracefully or becomes a bottleneck under load.

MODERNIZED ENCODER-DECODER ARCHITECTURE: FLASH ATTENTION T5

Encoder–decoder architectures can in some cases outperform pure decoder-only models, especially in multitask or zero-shot settings. [Flash Attention T5](#) (FAT5) is a modernized encoder–decoder model that updates the original T5 ¹ design for today’s efficiency standards. By integrating Flash Attention into T5’s architecture, it removes the main attention bottleneck, achieves near linear memory behavior, and supports significantly longer context windows. The model preserves strong generation capabilities while enabling efficient encoder-only fine-tuning for classification tasks, reducing training time without losing accuracy.

One approach is to reuse KV caches for shared prefixes across beams during beam search decoding to avoid redundant attention computations.

This can significantly cut memory usage and speed up inference, especially in large-beam setups. As the context grows, implement prefix trimming or a sliding window mechanism once the sequence approaches the model's maximum context size. Older tokens that no longer contribute meaningfully to the output can be safely dropped, reducing memory use. In addition to memory optimization, this is also a practical mitigation for the haystack problem, where too much historical context obscures relevant information.

SOFTMAX AND THE HAYSTACK PROBLEM

In transformers, attention weights are computed with Softmax. As the context grows, the denominator of Softmax, which sums over all token scores, becomes very large while each individual score stays roughly the same. This flattens the distribution, reduces contrast between important and irrelevant tokens, and makes it harder for the model to identify key information. This challenge is known as the haystack problem.

In deployment, you should continuously monitor cache hit ratio and memory pressure. Low hit ratios often reveal inefficient scheduling or unnecessary cache invalidations, while high memory pressure signals the need for paging or trimming strategies. Frameworks such as vLLM already support hierarchical paging and segment-level cache reuse to maintain stability under high concurrency. [Table 4-3](#) shows an overview of the most important vLLM features for helping you to manage your KV cache efficiently.

Table 4-3. vLLM Cache Management Features

Mechanism	Description	Benefit
Paged Attention	Splits the KV cache into fixed-size memory blocks managed by a <code>free_block_queue</code> ; the scheduler allocates and reclaims these blocks dynamically.	Enables efficient reuse, prevents fragmentation, and supports long-running sessions.
Prefix Caching	Hashes repeated token prefixes and reuses their cached K/V pairs for new requests with identical prefixes.	Eliminates redundant prefill computation for shared prompts or templates.
Chunked Prefill	Splits long prompt prefills into smaller chunks that are processed incrementally.	Prevents a single long prompt from monopolizing cache blocks and improves concurrency.
Continuous Batching	Schedules prefill and decode together based on real-time cache availability.	Increases throughput while keeping cache state consistent.
Hash-Based Verification	Compares stored token-block hashes with incoming ones before reusing a cache entry.	Ensures cache integrity after context changes, and avoids reusing stale or mismatched KV blocks when prefixes are trimmed or reordered.
Dynamic Cache Reclamation	Returns KV blocks to the pool immediately once a request completes or is preempted.	Keeps GPU memory utilization high and stable.
Disaggregated KV Transfer	Shares KV across prefill/decode workers (for example via a shared storage connector).	Supports distributed setups and keeps latency sensitive, decodes isolated from heavy prefills.

Another thing you would want to think about is how *context engineering*, the deliberate modification, insertion, or pruning of tokens in the model’s input context, directly affects KV cache integrity. Since the cache stores key and value projections of previously processed tokens, any modification that alters token order or identity may invalidate portions of it.

When new documents are injected, old prefixes are trimmed, or messages are reordered, cached entries no longer align with the model’s positional encodings. Reusing them leads to incoherent or hallucinatory outputs. Therefore, every structural context change must trigger selective

cache invalidation. A layered policy, that is combining windowing, segmentation, and hash verification, helps you to preserve correctness while maintaining efficiency. Intelligent context design, such as preallocating static system prompts, enables partial cache persistence even in dynamic RAG or multi-turn settings. Modern inference frameworks address this through:

- Sliding-window caching to retain only the most recent N tokens for bounded attention.
- Chunked or segment-level caching to isolate system prompts, retrieved context, and user turns for partial invalidation.
- Stale-key eviction which discards inactive caches after a timeout.
- Context hashing to compare token checksums before cache reuse to ensure correctness.

I'm sure, after reading this section, it's clear that deploying or making a decoder-only model faster isn't just about retraining, quantization or how many GPUs you have. It's about engineering the inference runtime and how you effectively manage its cache. And even though that might sound overwhelming at first, frameworks like vLLM simplify the process through paged memory, prefix reuse, and distributed KV sharing. In practice, you often only need to activate these mechanisms through the framework's configuration options, which lets you manage KV caches efficiently and with minimal operational overhead. Some functionalities, like prefix caching, are even activated by default.

Encoder-Only Models

This section switches your perspective to the often underappreciated encoder-only models. Decoder-only models are all about generation, producing one token after another. Encoder-only models, in contrast, are about understanding. They don't generate text, they analyze it. Their entire job is to look at the input from all directions and build a deep contextual understanding of what it means.

You've come across models like BERT² and RoBERTa³ before. These models read text bidirectionally, capturing relationships between words across the whole sequence. Instead of predicting the next token, they learn what's missing by reconstructing masked parts of the input. This approach, known as masked language modeling, teaches them to understand language structure and nuance rather than produce text.

Encoder-only architectures remove the decoder stack entirely and rely on bidirectional attention, which means every token can attend to every other token in the sequence. [Figure 4-2](#) illustrates an abstracted encoder-only architecture.

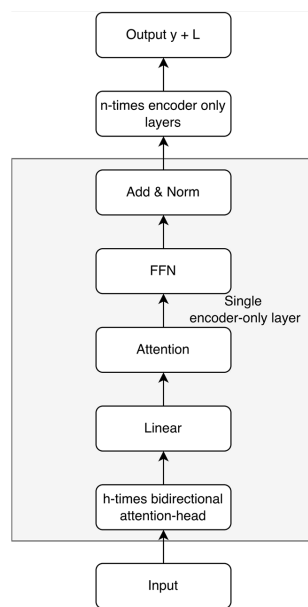


Figure 4-2. Abstracted encoder-only architecture.

Because there is no autoregressive loop, these models process entire inputs in parallel, making them faster, lighter, and cheaper to serve. They don't need key value caching or token by token generation, which makes them ideal for classification, retrieval, and embedding workloads.

Even as large generative models dominate headlines, encoder-only transformers continue to play a crucial role in real world systems. In RAG pipelines, for example, they act as retrievers, the fast and sharp analysts that surface the most relevant context before a generative model takes over. In large scale deployments, they are often the workhorses that enable efficiency and precision at scale. And the good news is that, even though most of the older architectures are still working fine, there exist modern adaption of the famous BERT model. [Table 4-4](#) shows newer model architectures, and how they improved over the original BERT in terms of positional encoding, sequence length and FlashAttention support for faster training and inference.

Table 4-4. Comparison of Encoder Model Architectures

Component	BERT (base)	BERT (large)	NomicBERT (base)	ModernBERT (base)	ModernBERT (large)	NeoBERT (medium)
Layers	12	24	12	22	28	28
Hidden Size	768	1024	768	768	1024	768
Attention Heads	12	16	12	12	16	12
Parameters	120M	350M	137M	149M	395M	250M
Positional Encoding	Positional Embeddings	Positional Embeddings	RoPE	RoPE	RoPE	RoPE
Sequence Length	512	512	2048	1024 → 8192	1024 → 8192	1024 → 4096
FlashAttention Support	✗	✗	✓	✓	✓	✓

NeoBERT⁴ is the latest modernization of the classic BERT architecture and performs extremely well on extended sequences. [Figure 4-3](#) compares the model throughput (tokens per second) as a function of sequence length (higher is better).

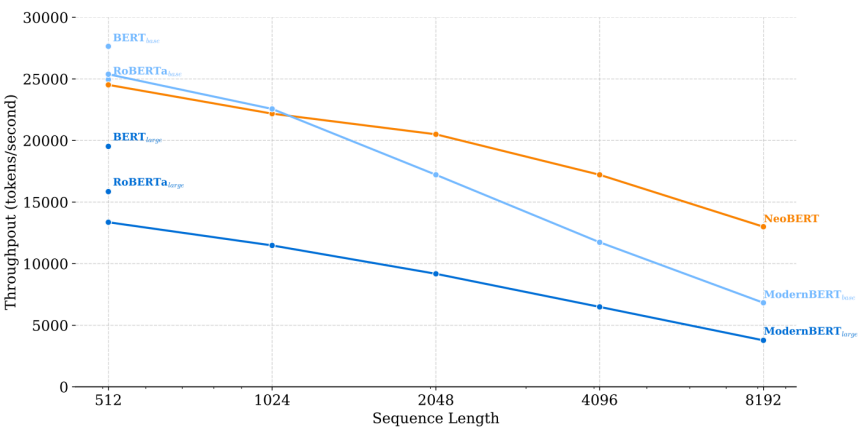


Figure 4-3. Throughput comparison of the two most recent modernized BERT architectures, ModernBERT and NeoBERT. Image is taken from NeoBERT, Lola Le Breton et al.

So, even though encoder-only models once seemed overshadowed by the rise of decoder-only architectures, their evolution has quietly continued. Modern variants such as [NomicBERT](#), ModernBERT⁵, and NeoBERT integrate SOTA techniques like rotary positional encodings for extended context windows and FlashAttention kernels for faster training and inference.

ROTARY POSITIONAL EMBEDDINGS (ROPE)

Traditional positional encodings, either fixed sinusoidal or learned, allow models to understand token order but fail to generalize well beyond their training context (for example, from 2k to 32k tokens). Rotary positional embeddings (RoPE) ⁶ address this limitation by encoding relative position through rotation rather than addition.

Instead of adding a positional vector to each token, RoPE rotates the query and key vectors in the attention mechanism by an angle that increases with token position. The dot product between two rotated vectors naturally reflects their relative distance, enabling the model to extrapolate to much longer sequences.

You can think of RoPE as arranging tokens around a circle: each word occupies a unique angle, and their angular difference represents relative distance. Nearby tokens align closely, while distant ones are separated by larger rotations. [Figure 4-4](#) illustrates this.

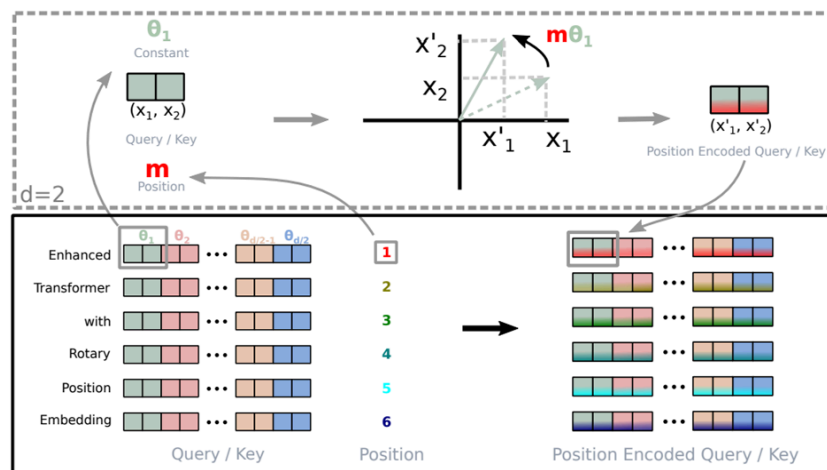


Figure 4-4. Illustration of Rotary Positional Embedding (RoPE). Image adapted from: Jianlin Su et al.

Extending RoPE-based models is straightforward. Modern libraries like vLLM or Transformers let you rescale RoPE frequencies to expand the context window at load time:

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="answerdotai/ModernBERT-base",
    trust_remote_code=True,
    rope_scaling={"type": "linear", "factor": 4.0}
)

output = llm.generate("Summarize this 64k-token document...",
                      SamplingParams(max_tokens=512))
print(output[0].outputs[0].text)
```

Here, `rope_scaling` increases the model's effective context length four-fold, from 32k to 128k tokens, allowing efficient long-sequence reasoning without retraining.

These innovations bring them closer to the efficiency and reasoning depth of their decoder-based counterparts while preserving the precision and bidirectionality that make them indispensable for retrieval, classification, and embedding tasks. As a result, encoder-only transformers remain central to modern AI systems, quietly powering the understanding layer beneath today's most advanced agentic systems.

This is why encoder-only models are your analysts and investigators within your team of AI agents. They don't speak: they listen, interpret and make sense of everything before passing it on. Their strength lies in understanding context deeply and efficiently, making them an essential part of any well-structured agent ecosystem.

Mixture of Experts Models

Now let's look at one architecture that changes how you can think about scale. MoE models take inspiration from how teams work in the real world. Not everyone needs to do every task. Some people specialize, and the challenge is to know whom to ask for help. MoE models apply that same logic to transformers.

Instead of having every layer process every token, which a dense model does, MoE models divide the work among multiple specialized networks called experts. These experts replace the standard Feed-Forward Networks (FFNs) found in dense transformer blocks. A lightweight gating network acts as a router that decides which experts to activate for a given input. In dense transformers, every token flows through the same FFN, while in MoE models the router selects only a few specialized FFNs. Only a small number of experts are used at a time, while the rest remain idle. This means the model can hold billions of parameters but only use a small fraction of them per forward pass. [Figure 4-5](#) illustrates this flow.

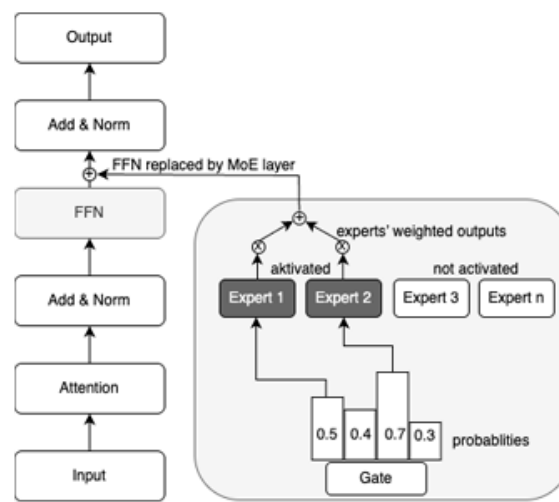


Figure 4-5. High-level overview of Mixture of Experts architecture.

The result is a model that scales capacity without scaling cost. A trillion-parameter MoE model can operate at a runtime cost similar to a much smaller dense model because it activates only the experts that matter. This concept is known as conditional computation: compute is spent only where it's needed.

The gating mechanism itself can vary. Common approaches include *top-k routing*, where the router activates the k most relevant experts; *noisy gating*, which adds stochasticity to improve expert utilization; and *expert-choice routing*, where experts bid for tokens instead of being assigned by the router. While these differences matter mainly during training rather than inference, they strongly influence load balancing, stability, and overall model quality.

Again, if you're thinking in team-member terms, MoE models are your specialists. They don't try to be good at everything, but when you route the right problem to the right expert, the system becomes both powerful and efficient. This makes MoE one of the most important architectural advances in large-scale language modeling, allowing teams of models to collaborate just as human experts would, each contributing their unique strength at the right moment.

Reasoning Models

If decoder-only models are your generators, encoder-only models your analysts, and Mixture of Experts your specialists, reasoning models are your thinkers. They are the ones who pause before they act, break problems into steps, and reflect on their own conclusions.

Reasoning models are not defined by architecture alone but by behavioral optimization. They're often decoder-based under the hood, but trained or prompted to use structured reasoning chains, planning traces, or tool-augmented reflection. Instead of predicting the next token direct-

ly, they learn to generate intermediate thoughts that guide better answers.

Some reasoning models, such as DeepSeek-R1, Qwen3, and OpenAI’s “reasoning” variants of GPT, extend this further with inference-time optimization: the model is encouraged to search its own reasoning space at runtime, sometimes generating multiple parallel solution paths before selecting the most consistent one. This shift marks the emergence of a new category, models that not only respond but reason dynamically.

Architecturally, they remain close to decoder-only transformers, but their real innovation lies in the inference loop rather than the network layers. They blend symbolic persistence (through reasoning tokens or hidden-state reuse) with probabilistic exploration (via multi-sample decoding or internal consistency checks). The result is slower generation, but deeper and more reliable decision-making.

In team-member terms, these are the colleagues who don’t rush to speak. They think, reflect, and sometimes even argue with themselves before offering an answer. They are slower, but they often save the whole team from avoidable mistakes. They are the reflective minds in an otherwise reactive system.

Thinking modes

Modern reasoning models support a control surface for how they reason. You can switch between a thinking mode and a fast response mode with deployment specific switches. With the Transformers library, you can enable or disable the internal reasoning trace directly in the chat template ([Example 4-1](#)).

Example 4-1. Switching between thinking and non-thinking mode with Transformers

```
text = tokenizer.apply_chat_template(  
    messages,  
    tokenize=False,  
    add_generation_prompt=True,  
    enable_thinking=True ❶  
)
```

❶ Enable thinking, which is the default mode.

With OpenAI compatible servers such as vLLM, you can pass mode switches through the client call, see [Example 4-2](#).

Example 4-2. Switching between thinking and non-thinking mode

```

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3-8B",
    messages=[
        {"role": "user", "content":
            "What are reasoning models, in terms of Large Language Models?"},
    ],
    max_tokens=8192,
    temperature=0.7,
    top_p=0.8,
    presence_penalty=1.5,
    extra_body={
        "top_k": 20,
        "chat_template_kwargs": {"enable_thinking": False}, ❶
    },
)

```

❶ {"enable_thinking": False} to disables thinking mode.

However, you can also “just tell” the model to not use thinking in the prompt. Even when thinking is disabled, many models keep an internal structure for the hidden trace, which preserves format consistency and allows you to toggle thinking without changing downstream code.

Use these switches to align latency and quality with the task. Enable thinking for planning, decomposition, tool selection, and verification. Disable thinking for short, low risk turns, or for steps where you already validated the plan.

Thinking budget

Reasoning tokens are not free. Many models expose a thinking budget that caps the number of tokens allocated to the internal trace. When the budget is exhausted mid-inference, the model can truncate its reasoning and continue the response gracefully. This can include a short handoff phrase that signals the budget boundary, as illustrated in [Example 4-3](#).

Example 4-3. Set thinking budet

```

thinking_budget = 1024
max_new_tokens = 32768

# Rest of code is omitted for brevity

if 151645 not in output_ids:

    if 151668 not in output_ids: ❶
        print("thinking budget is reached")
        early_stopping_text = "\n\nConsidering the limited time by the user,
            I have to give the solution based on the thinking directly now.\n</t

```

```

        early_stopping_ids = tokenizer([early_stopping_text], return_tensors=
        return_attention_mask=False).input_ids.to(model.device)
        input_ids = torch.cat([generated_ids, early_stopping_ids], dim=-1)
    else:
        input_ids = generated_ids
    attention_mask = torch.ones_like(input_ids, dtype=torch.int64)

    generated_ids = model.generate( ❷
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_new_tokens=input_length + max_new_tokens - input_ids.size(-1) ❸
    )
    output_ids = generated_ids[0][input_length:].tolist()

```

- ❶ Check if the thinking process has finished, `151668 is </think>`, then prepare the second model input for the second generation
- ❷ Second generation
- ❸ It might yield a negative result if `max_new_tokens` is too small, since the early stopping text consists of 24 tokens.

EVERY TOKEN COUNTS

Be mindful that each token counts, either if you're running the model self-hosted or via a managed API. Each reasoning step, tool invocation, and intermediate generation contributes to the final bill.

Choose the budget based on acceptable latency and task complexity. A budget that is too small can limit multi-step improvement and reduce consistency. In practice, values at or above one thousand tokens tend to work well for non-trivial reasoning, and you can raise the budget for math, code generation, and multi tool plans where deeper exploration pays off.

Putting It All Together: Designing a Team of Experts

By now you've seen how different model architectures bring unique strengths to your agentic systems. In this section you'll build a MAS based on this knowledge. Each agent in the following setup represents a distinct capability that contributes to the system's overall reasoning, retrieval, and synthesis process. [Table 4-5](#) shows the implementation with its roles and models.

Table 4-5. Agent team at a glance

Role	Architecture	Concrete model	Primary task	Tools it calls
Semantic research	Decoder only fast	Qwen3 32B	Neural search with highlights	External API via <code>exa.search_and_contents</code>
Patent research	Decoder only fast	Qwen3 32B	Past year patent hits	GoogleSerperAPIWrapper
Analyst	Hybrid controller plus encoder	Reasoning controller Qwen3 235B Thinking, Encoder ModernBERT base	Semantic filtering and extractive summary with layered reasoning then encoder scoring	<code>semantic_filter_tool</code>
Note taker	Decoder only fast	Qwen3 32B	Outline creation	<code>create_outline</code> , <code>read_document</code>
Writer	Decoder only fast	Qwen3 32B	Draft and edits	<code>write_document</code> , <code>edit_document</code> , <code>read_document</code>
Supervisor	Deliberate LLM	GPT 4.1	Routing and finish	agents as tools

Together, these specialized agents form a coordinated research and writing pipeline. The supervisor oversees the entire process, routing tasks between the agents and determining when the system has reached its final outcome. [Figure 4-6](#) illustrates how these components connect, highlighting the flow of information and reasoning within the multi-agent system.

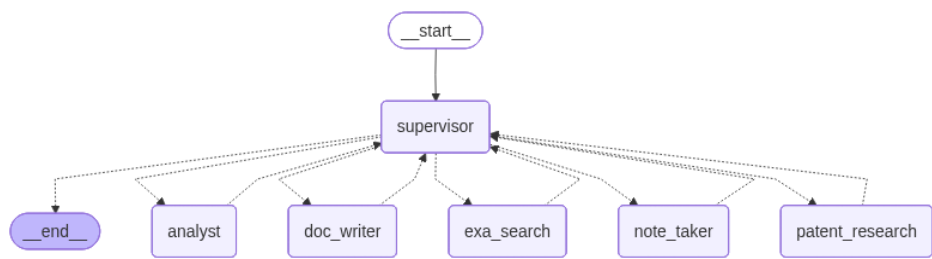


Figure 4-6. Illustration of the research team implemented in LangGraph.

The code in this section focuses on how each agent is defined. The complete code, including tools, graph wiring and routing, is intentionally

omitted so the focus remains on the division of roles and the architectural choices behind them.

[Example 4-4](#) defines the specific skill set for each agent in your team.

Example 4-4. Simple text generation

```
EXA_PROMPT = """Role: Research assistant. You can search for all recent info
on Exa Search. Your response should clearly articulate the key points you fo

PATENT_PROMPT = """Role: Market researcher with 20 years of experience.
You are very knowledgeable in patent research and in finding up-to-date info
about patents using the Google Patents API."""

NOTE_PROMPT = """You can read documents and create outlines for the document
writer. Don't ask follow-up questions."""

WRITER_PROMPT = """You can read, write and edit documents based on note-take
outlines. Don't ask follow-up questions."""

ANALYST_PROMPT = """
You are the Analyst. You must call the `semantic_filter_tool` exactly once.
Inputs:
- query: the current task topic or question
- documents: a list of raw strings gathered by teammates

Rules:
- don't answer the user
- don't do free form writing
- Only return the tool result to the supervisor
""".strip()
```

For the open source models I used [Nebius](#), but you can use any Open AI compatible cloud provider such as [Novita](#) or [Together AI](#). [Example 4-5](#) shows my models setup. You'll see that I deactivate the thinking of `Qwen3-32B`, and that I assign a thinking budget for `Qwen3-235B-A22B`. This is usually the same schema for common cloud providers and is adapted from the [Qwen documentation](#) on deploying the model via [vLLM](#).

Example 4-5. Setting up the models

```
fast_llm = ChatOpenAI(
    model="Qwen/Qwen3-32B-fast",
    temperature=0,
    api_key=NEBIUS_API_KEY,
    base_url="https://api.studio.nebius.ai/v1/",
    extra_body={
        "top_k": 20,
        "chat_template_kwargs": {"enable_thinking": False},
    },
)
```

```

thinker_llm = ChatOpenAI(
    model="Qwen/Qwen3-235B-A22B-Thinking-2507",
    temperature=0,
    api_key=NEBIUS_API_KEY,
    base_url="https://api.studio.nebius.ai/v1/",
    extra_body={
        "chat_template_kwargs": {"enable_thinking": True},
        "thinking": {"type": "token", "budget": 1536}
    },
)
supervisor_llm = ChatOpenAI(
    model="gpt-4.1",
    temperature=0
)

```

[Example 4-6](#) shows how to implement a helper function to create your agents faster.

Example 4-6. Create react worker node

```

def make_react_worker_node(
    *,
    llm: ChatOpenAI,
    name: str,
    tools: list,
    prompt: str | None = None,
    goto: str = "supervisor",
):
    agent = create_react_agent(llm, tools=tools, prompt=prompt)

    def node(state: State) -> Command[Literal["supervisor"]]:
        result: Dict[str, Any] = agent.invoke(state)
        msgs: Sequence[BaseMessage] = result.get("messages", [])
        content = getattr(msgs[-1], "content", "") if msgs else ""
        return Command(
            update={"messages": [HumanMessage(content=content, name=name)]},
            goto=goto,
        )

    return node

```

With this helper function, you can create your individual agents as show in [Example 4-7](#).

Example 4-7. Create agents

```

specs = [
    dict(
        name="search", tools=[tavily_search, tavily_extract],
        prompt=SEARCH_PROMPT, llm=fast_llm
    ),
    dict(

```

```

        name="exa_search", tools=[exa_search_tool],
        prompt=EXA_PROMPT, llm=fast_llm
    ),
    dict(
        name="patent_research", tools=[patent_search],
        prompt=PATENT_PROMPT, llm=fast_llm
    ),
    dict(
        name="analyst", tools=[semantic_filter_tool],
        prompt=ANALYST_PROMPT, llm=thinker_llm
    ),
    dict(
        name="note_taker", tools=[create_outline, read_document],
        prompt=NOTE_PROMPT, llm=fast_llm
    ),
    dict(
        name="doc_writer", tools=[write_document, edit_document, read_document],
        prompt=WRITER_PROMPT, llm=fast_llm
    ),
]

nodes = {
    s["name"]: make_react_worker_node(llm=s["llm"], name=s["name"],
    tools=s["tools"], prompt=s["prompt"])
    for s in specs
}

```

When you run the code and inspect the output of the analyst agent you'll see something like this:

```

Analyst (ModernBERT) processing 2 docs...
'analyst': {'messages': [HumanMessage(content='\n\n{\n  "text": "### Lat
Developments in AI Agents: A Focus on Coding Agents\n\nAI agents, particul
those focused on coding, are rapidly evolving and transforming the landscape
software development.",\n  "source": "Source Doc [1]",\n  "score": 0.956\n  },
# Rest of output omitted

```

You see in the second last line the score of 0.956, which is the relevance ranking score ModernBERT assigned to this particular content.

This setup gives you what you want: a single, deliberate analyst that reasons carefully over evidence, cross-checked by a modern encoder model, while everyone else remains fast and cheap. This is an example how to design self-auditing systems, by checking for relevance and potential hallucination in addition to prompting the model to provide the links to its resources.

Open vs. Closed Models: Why the

Choice Matters

There is a long-standing debate on whether one should use open or closed source models. But the open-vs-closed choice is not just a technical decision, it defines your company's strategic trajectory in building products with AI agents. When you look more closely, this choice affects everything from cost structure and compliance to innovation velocity, vendor dependence, and long-term data sovereignty. This section aims to help you to think about this from an analytical standpoint, by considering the following factors.

Speed of innovation and deployment

Contributions from the global research community drive new architectures and optimization techniques, while ecosystems such as Hugging Face, vLLM, or Ollama make these innovations accessible. In contrast, closed-source models progress at the pace of internal R&D cycles and scheduled releases. But it's not just this speed of innovation. You should also think about your own speed of innovation. That is your deployment speed adds nuance to this discussion. Closed models can be used instantly through APIs. Most open models can be accessed via managed cloud deployments, using providers like Together.ai, allowing rapid experimentation before teams eventually self-host for cost control and customization.

Accessibility

Open-source models are freely accessible for experimentation, inspection, and benchmarking. They often provide visibility into training data, architectures, and weights, which can be critical for auditing and compliance. Closed-source APIs restrict visibility and control, with limits on rate, cost tiers, and regional access. Accessibility also includes operational ease. Closed models remove infrastructure complexity and provide direct access, whereas open models might require infrastructure management.

Model optimization and customization

Open models still offer the broadest form of control. While you can fine-tune or customize a closed source model like GPT-4.1 via DPO or RL, this model exists still only inside the vendor's environment. You don't control the model weights, deployment, or runtime. In other words, customization is possible, but you are still vendor locked in. Open-source models let you carry your improvements with you; closed systems let you borrow flexibility, not own it.

Performance

Closed-source models often lead benchmark leaderboards due to proprietary compute budgets and training data advantages.

However, the performance gap has narrowed significantly. Modern open architectures such as Qwen-3 or Kimi K2 come close in performance to proprietary models. And with optimization techniques as you’ve read about in the previous chapter ([“The ART of Learning from Experience”](#), [“Enabling General-Purpose Agentic Programs Through Post-Training and Search-Based Agents”](#)), you can even train your models to improve their tool usage or collaborate better as a team.

Cost

Closed models can cost up to \$150 for input and \$600 (o1-pro) for output per million tokens, while open models fall around \$0.3 for input and about \$3 for output per million tokens when used via low-cost cloud providers even for models over 200B parameters. In addition, when you host them yourself, trade-off becomes operational. Open-source deployments require GPUs, scaling infrastructure, and ongoing maintenance. In the short term, managed APIs can appear cheaper because they remove the need for specialized staff, but in the long run, self-hosted open models usually yield a lower total cost of ownership once scaled efficiently.

Data security and compliance

When self-hosted or deployed on private cloud infrastructure, open models ensure full data ownership and compliance control, important for regulated industries like finance or healthcare.

Organizations can implement granular access control, audit mechanisms, and on-prem encryption layers. With managed models, either via closed or open models, you rely on vendor-managed security and compliance certifications (SOC-2, ISO-27001, etc.). This can simplify procurement but at the cost of reduced visibility and dependence on vendor diligence.

Given all of this, a hybrid strategy often makes the most sense. When you start to build your agentic system using managed closed APIs or managed open-source providers lets you move fast and focus on building. It keeps things simple while workflows and responsibilities are still forming. However, over time, as your team grows and your use cases begin to require more control or customization, it becomes more practical to bring open-source deployments in-house. With a larger team and clearer ownership of responsibilities, moving infrastructure and model management under your own roof often becomes the more effective path.

Optimizing Open Source Models

When you decide to go the open-source route, there are several ways to optimize both performance and cost. Traditionally, fine-tuning a large model meant updating all of its parameters as a continuation of the origi-

nal training process. For models with billions of parameters, this becomes expensive, since each trainable parameter adds roughly a fourfold memory overhead during optimization, and every checkpoint can occupy tens of gigabytes of storage.

Modern pipelines therefore rely on PEFT, a family of techniques that adapts large models by updating only a small subset of weights. The most common forms are LoRA and adapter modules, which specialize a base model for new tasks without retraining it from scratch, and quantization, which reduces memory and inference cost by lowering weight precision.

The following subsections walk you through these methods, serving as a practical guide to choosing the right optimization strategy when building and deploying agentic systems.

LoRA and Adapters

*Low-Rank Adaptation*⁷ (LoRA) and adapter modules are among the most popular methods to make fine-tuning more efficient. LoRA works by injecting low-rank matrices into existing weight layers, learning small corrective updates without touching the original model weights. Even though only a fraction of the parameters are updated, LoRA achieves performance comparable to full fine-tuning. During inference, the adapter weights are merged with the base model parameters so the system operates as a single unified model.

Adapters follow a similar principle by inserting small trainable layers between transformer blocks. This makes it possible to achieve near full fine-tuning performance at a fraction of the compute and memory cost. Instead of retraining the entire model, you insert additional lightweight modules that learn task-specific representations while keeping the base model frozen. This isolation helps prevent *catastrophic forgetting*, a phenomenon where a model loses previously learned capabilities when fine-tuned on new data. By preserving the original weights and only training a small number of additional parameters, adapters retain prior knowledge while learning new tasks. Both methods make it easy to switch between task-specific configurations by loading or unloading adapter weights as needed. This modularity is especially useful in production environments where a single model backbone must support multiple domains or languages.

A recent innovation that makes these ideas easy to implement is [LoRA Exchange](#) (LoRAX), which redefines how fine-tuned models are served in production. Instead of dedicating separate GPU resources to each LoRA adapter, LoRAX enables dynamic loading and batching of multiple fine-tuned models on a shared GPU. Through mechanisms such as dynamic adapter loading, tiered weight caching, and continuous multi-adapter

batching LoRAX helps you to pack over a hundred specialized LoRA models into a single deployment.

Dynamic adapter loading allows adapter weights to be fetched just-in-time at runtime, keeping latency minimal while eliminating the need to pre-load all fine-tuned variants. *Tiered weight caching* manages GPU, CPU, and disk memory to avoid out-of-memory errors, while *continuous multi-adapter batching* ensures requests for different adapters can be processed concurrently and fairly. Together, these strategies make it possible to serve many specialized adapters on a single GPU with one base model, helping you to lower your inference costs. For instance, a team could fine-tune different domain-specific models for customer support, legal analysis, or code generation, and deploy them all under one shared GPU pool, something that could otherwise cost tens of thousands of dollars in cloud resources.

Quantization

Quantization reduces model size and speeds up inference by representing weights with fewer bits, such as 8-bit or even 4-bit precision. This compresses the model's memory footprint, lowers GPU requirements, and enables faster inference. Frameworks such as [bitsandbytes](#) and [unsloth](#) simplify this process, offering robust quantization routines and optimized training kernels. In addition, many recent models released by developers such as Meta, Google, or Alibaba Cloud already include quantized variants by default, which makes deployment easier and more cost-efficient.

Usually, a well-optimized open-source pipeline often combines these techniques: quantized weights for efficient inference, LoRA or adapters for task specialization, and PEFT for adaptation. Together, they enable you to achieve high-quality results on smaller budgets and with shorter iteration cycles.

Conclusion

The central lesson of this chapter is that your agentic system's capability comes from the fit between task, model, and inference strategy. Models are team members with distinct strengths and skills. Decoder-only models are your generators and planners. Encoder-only models are your analysts and investigators. Mixture of Experts models are your specialists that add capacity without paying full compute on every token. Reasoning models are your deliberate thinkers, the ones that pause, reflect, and refine before acting. When you staff your system with this mindset, you get higher quality at lower cost.

You saw that architectural choice is as much an inference problem as a modeling problem. KV caching turns sequential generation into a

throughput win by reusing past keys and values, but it increases memory pressure. With reasoning models, you also need to decide when to enable thinking modes and how to set thinking budgets, so you spend compute where it matters and keep fast paths where it does not.

You composed these pieces into a working organization of agents. A deliberate controller that reasoned, an encoder that verified relevance, fast decoders for drafts and edits, all orchestrated by a supervisor that routed and decided when to finish the task.

You learned that open and closed model choices are not purely technical. They set your trajectory for cost, compliance, and data sovereignty. A hybrid path is often the pragmatic route. Start with managed APIs for quick iteration, then move to open-source self-hosting as your team and workflows mature.

Parameter-efficient fine-tuning lets you adapt models without touching every weight. LoRA and adapters provide modular specializations you can swap in and out per task or domain. Quantization shrinks memory and accelerates inference. Together with frameworks like bitsandbytes and unsloth, these techniques turn advanced optimization into a routine part of your build process.

Taken together, these ideas close the gap between architecture, inference, and operations. Choose the right roles, allocate thinking where it pays off, optimize the parts you own, and adopt a deployment strategy that matches your stage.

The next chapter shifts the focus from model architecture to system integration and execution. You will learn how to connect your agents to the outside world through structured tool calls, secure execution environments, and well-defined interface contracts.

- ¹ Colin Raffel et al. [“Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.”](#), (2019).
- ² Jacob Devlin et al. [“BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.”](#), (2019).
- ³ Yinhan Liu et al. [“RoBERTa: A Robustly Optimized BERT Pretraining Approach.”](#), (2019).
- ⁴ Lola Le Breton et al. [“NeoBERT: A Next-Generation BERT.”](#), (2025).
- ⁵ Benjamin Warner et al. [“Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference.”](#), (2024).

- 6 Jianlin Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding”, <https://arxiv.org/abs/2104.09864> (2021).
- 7 Edward J. Hu et al. “[LoRA: Low-Rank Adaptation of Large Language Models.](#)”, (2021).

Table of contents collapsed